Journal of Parallel and Distributed Computing 199 (2025) 105042

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

journal homepage: www.elsevier.com/locate/jpdc

# DRViT: A dynamic redundancy-aware vision transformer accelerator via algorithm and architecture co-design on FPGA

Xiangfeng Sun [a,1], Yuanting Zhang [a,1], Qinyu Wang [a], Xiaofeng Zou [a], Yujia Liu [b], Ziqian Zeng [a,*], Huiping Zhuang [a,*]

[a] South China University of Technology, Guangzhou, 511442, China
[b] Guangdong University of Technology, Guangzhou, 510006, China

ARTICLE INFO

*Keywords:*
Multi-modal artificial intelligence (MAI)
Vision Transformer (ViT)
Field-programmable gate array (FPGA)
Lightweight algorithm
Algorithm and architecture co-design

ABSTRACT

The multi-modal artificial intelligence (MAI) has attracted significant interest due to its capability to process and integrate data from multiple modalities, including images, text, and audio. Addressing MAI tasks in distributed systems necessitate robust and efficient architectures. The Transformer architecture has emerged as a primary network in this context. The integration of Vision Transformers (ViTs) within multimodal frameworks is crucial for enhancing the processing and comprehension of image data across diverse modalities. However, the complex architecture of ViTs and the extensive resources required for processing large-scale image data pose high computational and storage demands. These demands are particularly challenging for deploying ViTs on edge devices within distributed frameworks. To address this issue, we propose a novel dynamic redundancy-aware ViT accelerator based on parallel computing, termed DRViT. DRViT is supported by an algorithm and architecture co-design. We first propose a hardware-friendly lightweight algorithm featuring token merging, token pruning, and an INT8 quantization scheme. Then, we design a specialized architecture to support this algorithm, transforming the lightweight algorithm into significant latency and energy-efficiency improvements. Our design is implemented on the Xilinx Alveo U250, achieving an overall inference latency of 0.86 ms and 1.17 ms per image for ViT-tiny at 140 MHz and 100 MHz, respectively. The throughput can reach 1,380 GOP/s at peak, demonstrating superior performance compared to state-of-the-art accelerators, even at lower frequencies.

## 1. Introduction

In recent years, multi-modal artificial intelligence (MAI) has garnered considerable attention for its ability to process and integrate data from various modalities, such as images, text, and audio. Among the advancements in this field, the Transformer-like network has become the main network for handling MAI tasks in parallel processing and distributed computing environments. For instance, ViLT [1] integrates vision and language tasks by combining vision and language transformers, excelling in image-text matching and visual questioning tasks. Similarly, CLIP [2], proposed by OpenAI, utilized a vision transformer to encode images and a text transformer to encode text, creating a shared representation space through contrastive learning, thus enabling the model to understand and generate co-modal content. Among these models, the incorporation of Vision Transformers (ViTs) in multi-modal frameworks enhances the processing and understanding of image data across multiple modalities. Because ViTs outperform Convolutional Neural Networks (CNNs) by addressing the lack of inductive bias and enabling effective transfer learning in downstream tasks. ViTs also support parallelization across input sequences, offering higher efficiency and using approximately 2-4 times less computational power to achieve performance on par with CNNs [3–7]. However, distributed real-time computer vision tasks such as affect analysis with vision and speed [8], multi-modal human-computer interaction [9], and multi-modal healthcare [10] demand low latency, high security, and high energy efficiency. Cloud deployment often fails to meet these requirements, making edge devices a potential solution. But the limited computational resources on edge devices pose a notable challenge for achieving low-latency and energy-efficient parallel accelerators, which are crucial for these applications.

* Corresponding authors.
*E-mail addresses:* zqzeng@scut.edu.cn (Z. Zeng), hpzhuang@scut.edu.cn (H. Zhuang).
[1] The two authors contribute equally to this work.

https://doi.org/10.1016/j.jpdc.2025.105042
Received 30 June 2024; Received in revised form 27 December 2024; Accepted 25 January 2025

Available online 28 January 2025
0743-7315/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

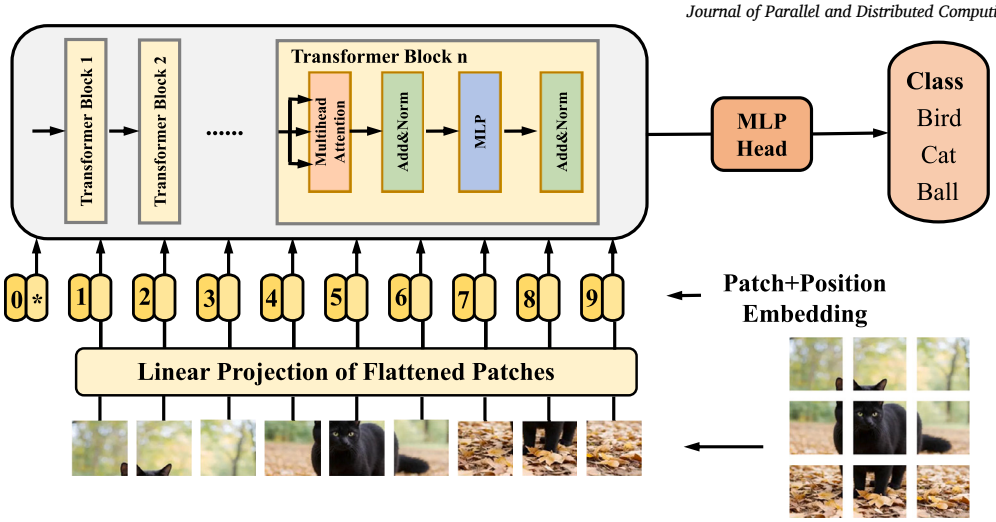


**Fig. 1.** Structure of ViT model [3]. ViT divides an input image into patches, flattens them, and adds positional embeddings. These patches are passed through Transformer blocks that apply self-attention mechanisms to capture relationships across the image. The final output is processed by an MLP head to classify the image into predefined categories.

One of the primary challenges is obtaining low latency, largely due to the multi-head self-attention mechanism. Firstly, in transformer-like networks, the self-attention mechanism identifies important words for a given query word in a sentence. This is done by passing the query ($\boldsymbol{Q}$), key ($\boldsymbol{K}$), and value ($\boldsymbol{V}$).

The detailed function is shown in Section 3. Unlike standard matrix multiplication, where the weight parameters can be pre-processed, both $\boldsymbol{K}$ and $\boldsymbol{V}$ are generated on-line, significantly increasing latency. Secondly, the multi-head data are high-dimensional tensors, further increasing computational complexity and time consumption on resource-limited edge devices. Thirdly, the Transformer architecture also relies heavily on various nonlinear and floating-point functions, including LayerNorm, Softmax, and GELU activation [3,5,11–13]. LayerNorm is applied to each layer after the self-attention mechanism or the Multi-Layer Perceptron (MLP) sublayers, normalizing the output of each layer to have zero mean and unit variance. Softmax scales numbers in a vector to the probability range of 0-1 and ensures their sum is equal to 1. GELU activation, based on the Gaussian error function, provides a smooth bell-shaped curve commonly used in neural networks.

Another important challenge is achieving energy efficiency. The energy cost of accessing the off-chip memory is much higher than accessing the on-chip buffer and computing, accounting for the main power consumption [14]. Unlike CNNs that leverage a single convolution kernel filter capable of sliding through the entire image with high data reuse times (typically ranging from 100 to 10,000) [11], Transformer-like networks exhibit relatively small reuse times of approximately several hundred. This leads to a considerable amount of memory access, significantly decreasing energy efficiency.

To reduce computational complexity and memory access, we propose a lightweight hardware-friendly algorithm to reduce computational workload in ViT by reducing redundancy. The proposed algorithm merges similar tokens, and prunes unimportant tokens, employs an INT8 quantization scheme. While this reduces the computational workload, it incurs irregular computing patterns that can hinder efficient architecture design. To address this, we propose a novel dynamic redundancy-aware specialized architecture based on a field-programmable gate array (FPGA), termed DRViT, turning token merging, token pruning, and INT8 quantization into latency and energy efficiency improvements. DRViT is designed to support the proposed hardware-friendly lightweight algorithm efficiently and mitigate these irregularities.

To summarize, the main contributions of our work are as follows:

- We propose a lightweight hardware-friendly algorithm with token merging, token pruning, and INT8 quantization, reducing power consumption and latency.
- We develop a dynamic redundancy-aware hardware architecture to support the lightweight algorithm, achieving high energy efficiency and performance through algorithm and architecture co-design.
- The proposed methodology is generalizable and can be extended to a range of Transformer-like accelerators. The design, implemented on the Xilinx Alveo U250 FPGA, achieves an overall inference latency of 0.86 ms and 1.17 ms per image for ViT-tiny at 140 MHz and 100 MHz, respectively. The throughput reaches 1380 GOP/s at peak, demonstrating 2.24× higher performance improvements over previous state-of-the-art accelerators.

The remainder of this article is structured as follows. Section 2 provides background on the ViT algorithm and FPGA accelerators for ViT. Section 3 describes the newly developed lightweight hardware-friendly algorithm proposed in this study. Section 4 highlights the innovative, specialized architecture designed for this lightweight ViT algorithm. Lastly, Section 5 presents the results of the implemented accelerator and compares them with previous work.

## 2. Background and related work

### 2.1. Vision transformer

The Transformer architecture was initially developed for natural language processing to address the complexities of modeling dependencies in one-dimensional sequence data. This process involves mapping individual words to feature vectors through an embedding mechanism and encoding these vectors layer by layer using an encoder-decoder architecture, crucial for capturing contextual relationships. In the realm of computer vision, the Vision Transformer (ViT) model [3] adopts a similar approach. It simplifies the architecture by omitting the decoder unit and dividing image data into fixed-size patches, thereby reducing the computational burden associated with computing similarities across all pixel features. As illustrated in Fig. 1, after position embeddings, the data is processed sequentially by $n$ transformer blocks within the transformer encoder. These blocks utilize techniques such as layer normalization, multilayer perceptron (MLP), multihead self-attention (MSA), and GELU. Finally, the transformer's output is processed by the MLP to produce the desired results. The success of ViT has spurred the

development of other Transformer-like networks, including the Swin Transformer [15] and DeiT [5].

### 2.2. Lightweight vision transformer

The ViT model excels in image classification and object detection tasks [16], yet its extensive parameter count and computational demands challenge its deployment on edge devices with limited resources. Quantization has emerged as a potential solution to this issue, considerably reducing the model's computational and memory footprint by converting floating-point operations to lower precision formats [17]. For example, the INT8 quantized ViT-s model with 21.4 million weight parameters, achieves a Top-1 accuracy of 81.27%. Similarly, the tinyViT model [18] attains a Top-1 accuracy of 73.0% using INT8 quantization. However, most prior research has focused on software-based simulation quantization, often termed "fake quantization", where only the inputs and outputs use integer values, while the compute-intensive operations still rely on floating-point values [11]. Another approach to make the algorithm lightweight involves token merging and pruning.

Token merging consolidates similar tokens to streamline the algorithm. The Token Clustering Transformer (TCFormer) [19] gradually merges tokens in multiple stages, adapting to different images, while Token Merging (ToMe) [20] uses a lightweight matching algorithm for merging. Token pruning, on the other hand, reduces computational costs by eliminating redundant tokens. For instance, Dynamic Token Sparsification ViT (DynamicViT) [21] dynamically prunes redundant tokens on a layer-by-layer basis, and Adaptive Tokens for Efficient Vision Transformer (A-ViT) [22] adjusts the inference cost adaptively for different images. However, a common drawback of these methods is the retention of background tokens, it can interfere with image recognition results. To mitigate this, TMTP [23] combines token merging and pruning, effectively reducing computational complexity without compromising accuracy. Despite these advances, the design is optimized for GPU platforms and may not be suitable for resource-limited edge devices, underscoring the need for further research to optimize ViT models for edge deployment.

### 2.3. FPGA accelerator of vision transformer

The development of hardware accelerators for ViT has been a subject of recent works [24,25], presenting two main technical approaches [26–28]: the stream pattern and the overlap pattern. The stream pattern consists of modules connected sequentially according to the target algorithm, where each module's output feeds into the next module, facilitating parallelism and optimization based on the specific computational characteristics of each module. This design pattern enables the implementation and parallel scheme optimization in each hardware kernel, according to the differences in computing characteristics. However, its lack of flexibility is a disadvantage [11]. On the other hand, the control pattern is based on the completion of all computing on a unified computational unit, with a control unit coordinating the entire process. This approach can accelerate various models on the same accelerator, but its one-size-fits-all design may lead to inconsistent performance on network models with different workload characteristics [29,30]. Given the diverse computational characteristics inherent in ViT models, the stream pattern emerges as a more suitable option due to its ability to optimize and parallelize operations effectively.

## 3. Hardware-friendly algorithm

In this section, we introduce a lightweight hardware-friendly algorithm featuring dynamic token merging, token pruning, and an INT8 quantization scheme, greatly reducing the redundant computational workload for ViT.

### 3.1. Algorithm overview

Although ViT has achieved notable success in computer vision, its high demand for computation and memory resources poses serious challenges for inference on resource-constrained edge devices. Two main factors contribute to the computational and memory resource demands of ViT models. Firstly, ViT models use image patches as input tokens [31], leading to redundant computations. Token merging and token pruning are potential solutions to this problem [32,33]. Secondly, the design of nonlinear and floating-point functions such as LayerNorm and Softmax contributes to computationally intensive operations. Integer quantization offers a promising solution to mitigate these computational demands [34]. Therefore, we propose a lightweight hardware-friendly algorithm with token merging, token pruning, and an INT8 quantization scheme.

For token merging, we introduce a novel method to evaluate the similarity between tokens, designed to be hardware-friendly. For token pruning, we utilize the class token attention score, which represents the correlation between a token and the class token, to determine token importance and avoid unnecessary computations. Our INT8 quantization scheme minimizes the use of intensive floating-point calculations, thereby reducing computational complexity and enhancing performance on hardware platforms. The details are as follows.

### 3.2. Token merging

Token merging is a crucial step for reducing the computational burden in ViT models by merging similar tokens based on their feature similarity. Specifically, we compute the similarity between tokens by measuring the distance between their feature vectors, which serves as the basis for merging decisions [20]. TMTP [23] proposes a method based on the dot product similarity of token key in a Transformer. However, while effective, this method is resource-intensive, particularly in edge devices, due to the energy consumption involved in calculating the dot products across all tokens.

Our approach tokenizes the input by dividing the input image into fixed-size patches, with each patch mapped to a token embedding. These token embeddings are processed through the Multi-Head Self-Attention (MHSA) mechanism at the attention layer. The attention mechanism computes the following operation for each token pair:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}, \tag{1}$$

where $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ represent the query, key, and value matrix, while $d_k$ is the token embedding dimension. $\boldsymbol{K}$ indicates the importance or similarity of each token based on its attention weight. Instead of calculating the full dot product matrix, we propose a hardware-friendly similarity estimation technique by selecting representative elements from token pairs and comparing matching features. This method significantly reduces computational overhead, making it suitable for deployment on edge devices.

Once token similarity is estimated, highly similar tokens are merged using a bipartite soft matching strategy [20]. This approach ensures that only tokens with strong feature similarity are combined, preserving the model's accuracy while reducing computation.

### 3.3. Token pruning

Token pruning is another optimization technique aimed at reducing computation by eliminating tokens deemed unimportant. In our proposed method, the importance of a token is determined by its class token attention score [23]. Given an input sequence of $n$ tokens in the ViT encoder, each token contributes to the final classification decision, with the class token playing a critical role. The attention score for the class token ($\boldsymbol{A}_{\text{cls}}$) is calculated using the following equation:

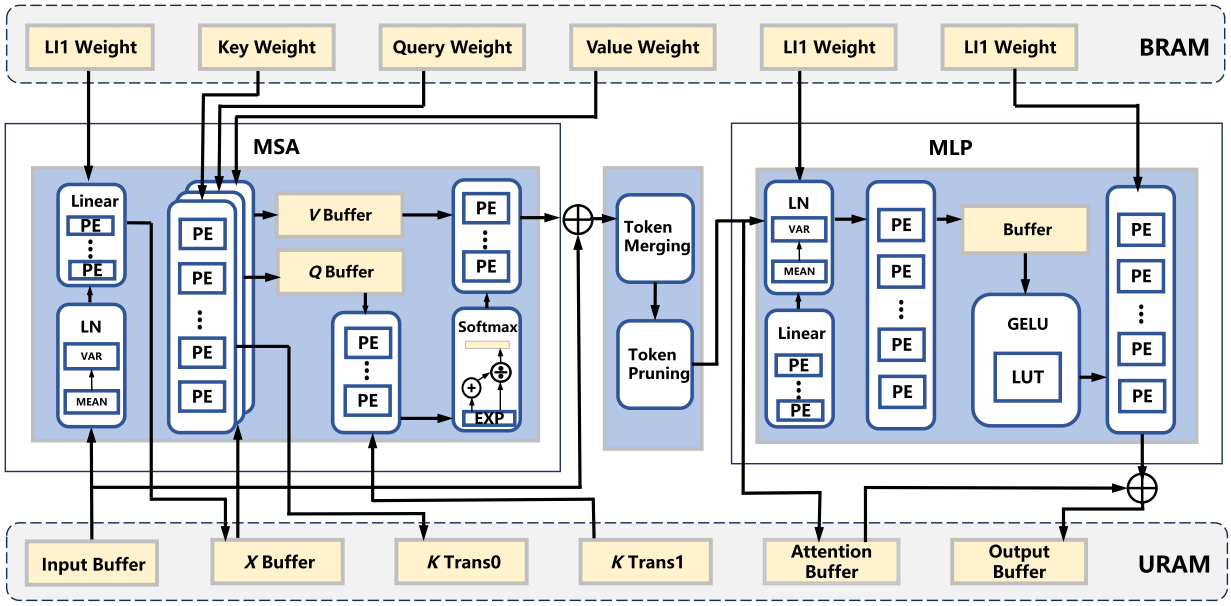


**Fig. 2.** Architecture overview of DRViT. Left block: multihead self-attention (MSA) block; Right block: multilayer perceptron (MLP) block; Mediate block: token merging and pruning; Up block: BRAM, utilized as ROM; Down block: URAM, utilized as buffer. Because ViT has multiple processing functions. The overall architecture employs the stream pattern. It implements different hardware units for each computing part in the calculation flow of the target algorithm and optimizes them separately to take advantage of interblock parallelism. Meanwhile, the on-chip flow greatly reduced memory access.

**Table 1**
Accuracy of various models INT8 quantification (Transposed and Adjusted). It is clear that INT8 quantization can achieve high accuracy for various models by this method.

| | ViT-S | ViT-B | DeiT-T | DeiT-S | DeiT-B | Swin-T |
|---|---|---|---|---|---|---|
| FP32 | 81.39% | 84.53% | 72.21% | 79.85% | 81.85% | 81.35% |
| INT8 | 81.27% | 84.76% | 72.24% | 80.12% | 81.74% | 81.50% |
| Diff | -0.12% | +0.23% | +0.03% | +0.27% | -0.11% | +0.15% |

$$\boldsymbol{A}_{\text{cls}} = \text{softmax}\left(\frac{\boldsymbol{q}_{\text{cls}}\boldsymbol{K}^{\text{T}}}{\sqrt{d_k}}\right), \tag{2}$$

where $\boldsymbol{q}_{\text{cls}}$ represents the query vector of the class token, and $\boldsymbol{K}$ is the key matrix of all tokens. A high $\boldsymbol{A}_{\text{cls}}$ value indicates a strong correlation between the token and the class token, signifying its importance in the classification task.

To prune unimportant tokens, we establish a dynamic threshold based on the class token attention score. Tokens with attention scores lower than this threshold are pruned, allowing the model to focus on more relevant information. The threshold can be adjusted to balance the trade-off between computation and accuracy, providing flexibility for different use cases. This pruning mechanism reduces computational complexity without introducing additional latency since the attention scores are already computed as part of the standard attention operation.

### 3.4. INT8 quantization

Quantization, specifically INT8 quantization, is employed to address the computational challenges associated with floating-point operations in ViT models, such as those required for LayerNorm and Softmax. I-ViT [35] demonstrates that INT8 quantization can maintain high accuracy across various models while substantially reducing computational load. Table 1 shows the minimal accuracy loss achieved using INT8 quantization.

While the current quantization approach is optimized for GPU platforms, we extend it to edge devices by implementing INT8 quantization on FPGA hardware. This shift diminishes costly floating-point operations, significantly reducing the model's computational complexity and memory footprint. Our INT8 quantization method assigns specific bit allocations for data representation: the first bit denotes the sign, the second to fifth bits represent the integer part, and the remaining bits handle the fractional part. This results in a minimum resolution of 0.125 for quantized data, effectively supporting model computations in resource-constrained environments. Moreover, the largest and smallest representable numbers are 15.875 and -15.875, respectively, with the special binary value "10000000" reserved for control purposes, such as initialization or error signaling. The details are presented in Section 4.3. Although our accuracy is slightly lower than the method in [5], our design demonstrates superior efficiency: the latency of the critical nonlinear function LayerNorm is reduced by 13.5%, and the total latency is reduced by 5.2%. This improvement highlights the balance between computational efficiency and model accuracy achievable through our quantization strategy.

## 4. Hardware architecture

In this section, we propose DRViT, a novel dynamic redundancy-aware specialized architecture for the lightweight ViT algorithm proposed. We introduce the overall stream pattern and key modules in our design including the Token Merging and Pruning Module, Integer-Only Nonlinear Function Module, Super Logic Region (SLR) Management Module, and so on. Based on the Token Merging and Pruning module, dynamic redundancy-awareness is achieved in the hardware architecture efficiently. The details are as follows.

### 4.1. Overall architecture

Fig. 2 illustrates the overall architecture of DRViT. The overall design adopts a stream structure. Because ViT with multiple processing functions, is more suitable for stream patterns [36]. It implements different hardware blocks for each computing part in the calculation flow of the target algorithm and optimizes them separately to take advantage of interblock parallelism [37,38]. To support the proposed lightweight algorithm and conserve limited transmission resources on edge devices, we use each token matrix as the transmission unit between neighboring blocks in our stream structure, facilitating easier merging and pruning

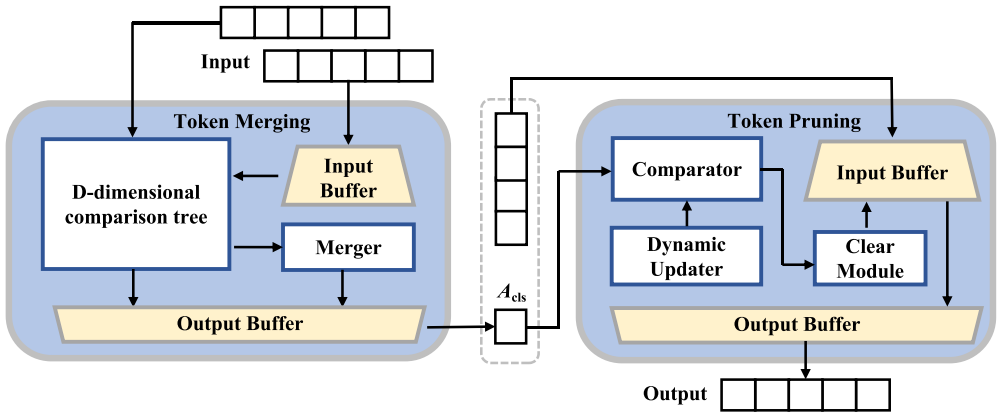


**Fig. 3.** The hardware design of the token merging and pruning module where the top module merges the similar tokens and the bottom module prunes the unimportant tokens.

on hardware. Additionally, to reduce latency and avoid extra power consumption, it is essential to ensure that the execution time of each block is similar. We define the execution time of each block as an "instruction cycle". At the end of each "instruction cycle", each block transmits one token matrix to the next block and receives one token matrix from the previous block. For ViT-tiny, we set the execution time of a vector multiplication as the baseline, with an "instruction cycle" of 216 clocks.

The accelerator primarily includes the Token Merging and Pruning Module, Integer-only Nonlinear Function Module, SLR Management Module, and Matrix Multiplication Module. The main functions of these hardware units are as follows: the Token Merging and Pruning Module efficiently prunes unimportant tokens and merges similar tokens efficiently in the stream pattern. The Nonlinear Function Module performs nonlinear operations in ViT, such as LayerNorm and GELU. The SLR Management Module manages the interaction between different SLRs and reduces the probability of timing problems occurring between different SLRs. The Matrix Multiplication Module achieves efficient matrix multiplication with reduced hardware area.

### 4.2. Token merging and pruning module

To support the token merging and pruning in the proposed lightweight algorithm efficiently, we present an efficient architecture design without incurring notable irregular computing patterns. The Token Merging and Pruning Module is located between the Attention module and MLP module, as shown in Fig. 2. In our stream pattern, a token matrix is a transmission unit between different modules, so it is very convenient to merge and prune tokens on hardware. Fig. 3 illustrates the hardware architecture of the Token Merging and Pruning Module. For token merging, the similarity comparison module compares the $\boldsymbol{K}$ values of all tokens through the D-dimensional comparison tree to represent the similarity between tokens. At the same time, the fusion module merges similar tokens in parallel and beats the output to fill in misaligned timings. For token pruning, it consists of a comparator, a dynamic update module, and a clearing module. When a token is an input and the comparator detects that its class token attention score is lower than the standard, the clearing module replaces it with an all-zero token, preventing irregular computing patterns in the overall stream structure. The dynamic update module will be set based on different models different standard class token attention scores are used to obtain the optimal pruning solution.

### 4.3. Integer-only nonlinear function module

#### 4.3.1. Softmax module

In the realm of deep learning and neural networks, the Softmax function plays a crucial role, particularly in classification tasks. In the Transformer, Softmax translates the attention scores into probabilities, as shown on Eq. (3)

$$\mathrm{softmax}(\boldsymbol{x}_i) = \frac{e^{x_i}}{\sum_j^n e^{x_j}} \ . \tag{3}$$

There are mainly two challenges for the calculation of Softmax. The first is that exponential arithmetic is typically unsupported by integer-only logic. The second is that it is a two-phase operation where division must be performed only after the completion of all data accumulation. To maintain efficiency in our design, we adopt a streaming pattern, necessitating that the throughput of each stage is similar and achieving the highest throughput is not required. Instead, a balanced throughput across stages is more critical. For ViT-tiny, the input size is (197,1), so the parallelism required for Softmax computation should be 2.

For the first phase, we use a two-way lookup table to achieve fast computation of exponential arithmetic. The two-way lookup table comprises 256 entries to cover 198 data, each precomputed to a precision of 8 bits. This table-based computation method offers low latency and high accuracy, thereby reducing the computational load during real-time processing. Using the two-way lookup table, we retrieve two adjacent data items in one clock cycle. Then, we place the first data item into the odd accumulator and the second data item into the even accumulator. Thus, the odd accumulator sums the $1^{st}$, $3^{rd}$, ..., and $197^{th}$ data items, while the even accumulator sums the $2^{nd}$, $4^{th}$, ..., and $198^{th}$ (with the $198^{th}$ item being 0) data items. Finally, we add the results of the two accumulators to obtain the total sum.

In the second phase, a two-way divider processes the accumulated results. Temporary storage is managed using two UltraRAMs, ensuring that data is available for division operations. The two corresponding dividers facilitate these calculations, maintaining consistent throughput across exponential value computation, accumulation, and division. Each phase requires 99 clock cycles, totaling 198 clocks, with an additional 18 clocks for redundancy to ensure accuracy, resulting in a complete instruction cycle of 216 clocks.

#### 4.3.2. LayerNorm module

Layer normalization (LayerNorm) is extensively used in Transformers to enhance model training and performance [39]. In ViT, LayerNorm is applied after the self-attention mechanism and MLP sub-layers, normalizing the output of each layer to zero mean and unit variance. This reduces input distribution shifts during training, improving training speed and stability, and mitigating the impact of outliers, thus enhancing model robustness.

The LayerNorm operation is given by:

$$\mathrm{LayerNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x} - \mathrm{Mean}(\boldsymbol{x})}{\sqrt{\mathrm{Var}(\boldsymbol{x}) + \varepsilon}}\gamma + \beta, \tag{4}$$

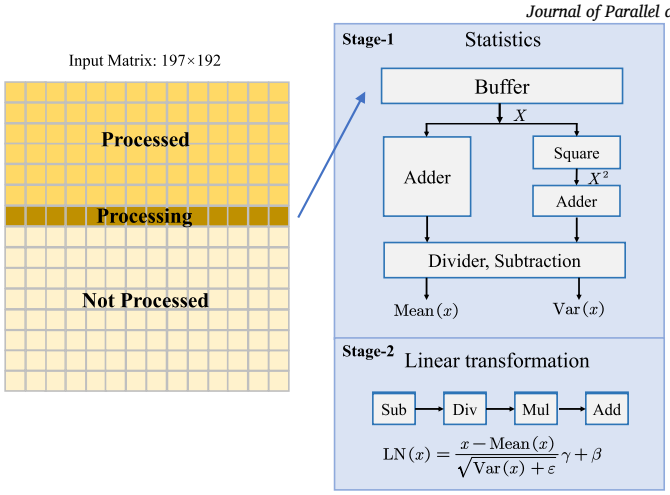


**Fig. 4.** The hardware design of LayerNorm module where stage-1 is to get the statistics mean and variation, stage-2 is to calculate the LayerNorm. (Yellow matrix: processed, Brown matrix: processing, Light yellow matrix: not processed.)

| | | | | | |
|---|---|---|---|---|---|
| $\mathrm{Sum1} = \mathrm{Sum1} + x_1$ | $\mathrm{Sum1} = \mathrm{Sum1} + x_2$ | ..... | $\mathrm{Sum1} = \mathrm{Sum1} + x_{192}$ | $\mathrm{E}(x) = \mathrm{Sum1}/192$ | $[\mathrm{E}(x)]^2 = \mathrm{E}(x) \times \mathrm{E}(x)$ |
| ${x_1}^2 = x_1 \times x_1$ | ${x_2}^2 = x_2 \times x_2$ | ..... | ${x_{192}}^2 = x_{192} \times x_{192}$ | | |
| | $\mathrm{Sum2} = \mathrm{Sum2} + {x_1}^2$ | $\mathrm{Sum2} = \mathrm{Sum2} + {x_2}^2$ | ..... | $\mathrm{E}(x^2) = \mathrm{Sum2}/192$ | $\mathrm{Var}(x) = \mathrm{E}(x^2) - [\mathrm{E}(x)]^2$ |

**Fig. 5.** Pipeline in LayerNorm module to get the statistics mean and variation. Yellow: accumulation for the input value $x$; Blue: square block to calculate $x^2$; Green: accumulation for the value $x^2$.

where "Mean($\boldsymbol{x}$)" and "Var($\boldsymbol{x}$)" represent the statistics of mean value and variance, respectively. It is also a two-phase operation, Mean and Var are needed to be calculated first. For the first phase, in order to improve computational efficiency, the mean and variance can be calculated in parallel. Mathematically, variance can be calculated using the following formula:

$$\mathrm{Var}(\boldsymbol{x}) = \mathrm{E}(\boldsymbol{x}^2) - [\mathrm{E}(\boldsymbol{x})]^2, \tag{5}$$

where $\mathrm{E}(\boldsymbol{x})$ is the mean value and $\mathrm{E}(\boldsymbol{x}^2)$ is the mean of squared $\boldsymbol{x}$. In the second phase involves parallel operations of division, multiplication, and addition for each data point.

Fig. 4 illustrates the hardware architecture of the Layer Norm part, which mainly consists of two modules: a statistics module and a linear transformation module. For ViT-tiny, these two modules each require one instruction cycle, that is 216 clock cycles, to compute a set of data. They combine to form a two-stage pipeline, as shown in Fig. 4. Firstly, the main function of the statistics module is to complete the statistical calculation of the mean and variance of this set of data. To solve the mean, an accumulator is needed to sum up each data item (i.e., $x$ in the formula). After summing, to calculate the average, division by the total number (192) is needed. We convert the division operation into multiplication with the reciprocal: (65536/192) times sum, resulting in 341 times sum. Additionally, to solve the variance, another accumulator is needed to sum up the squares of each data item (i.e., $\boldsymbol{x}^2$ in the formula) and calculate the mean in the same way. The difference between the two means is the variance. Thus, the total delay is 198 clocks. Secondly, the main function of the linear transformation module is to normalize the data based on the computed mean and variance. This process mainly has four steps, as shown in Fig. 5. These temporary data often involve negative numbers, requiring the use of two's complement for calculations. The first and fourth steps are performed using adders with two's complement functionality. For the second step, we designed a divider with two's complement functionality. Similarly, a multiplier with two's complement functionality is used for the third step. Thus, the total delay is 196 clocks.

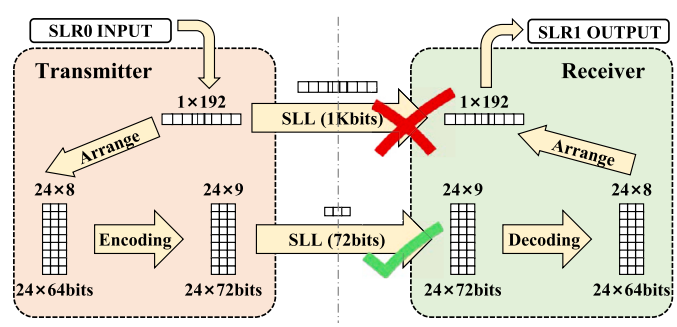


**Fig. 6.** Super Logic Region (SLR) Management Module. The left part represents the data transmitter, SLR0, and the right part represents the data receiver, SLR1. The two are connected via a super long line (SLL). Due to the bandwidth limitations of the SLL and the possibility of transmission errors, the raw data cannot be transmitted directly. Instead, the data is arranged and encoded, allowing it to be split into multiple parts for transmission.

### 4.4. SLR management module

FPGAs with abundant logic resources, such as the Alveo U250, are composed of multiple Super Logic Regions (SLRs). Specifically, the hardware resources in the Alveo U250 are divided into four SLRs: SLR0 through SLR3. Although each SLR has slightly differing resources, the variation is minimal, with the maximum difference being only 2.4%. This slight disparity does not impact the overall circuit design, so the four SLRs can effectively be treated as identical. Different SLRs may operate with different clock domains, and data exchange between SLRs relies on Super-Long Lines (SLL). At higher frequencies, this can result in timing issues and data transmission errors. To address this, we have developed an SLR Management Unit designed to synchronize the operation of the four SLRs and coordinate data transmission effectively.

In the context of a row-wise data flow consisting of 192 elements, and assuming INT8 quantization, the data size amounts to 1,536 bits.

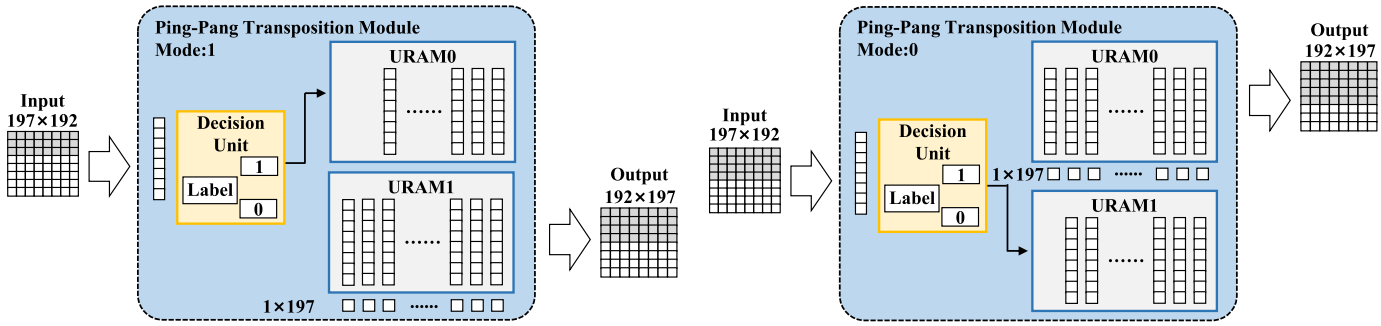


**Fig. 7.** The hardware design of the Ping-Pang module for $K$ matrix transposition. Mode0 (Left): the even matrix is written to URAM0 and the data in URAM1 is read; Mode1 (Right): the odd matrix is written to URAM1 and the data in URAM0 is read.

When data is transmitted from one SLR to another, the limited number of available Super-Long Lines (SLLs) can cause timing issues and data transmission errors, which impede the parallel transfer of these 1,536 bits. To resolve this, we introduce the SLR Management Module, as shown in Fig. 6, which is divided into two main components: the Transmitter and the Receiver. Each component performs two key functions:

Data Arrangement: Given the hardware constraints of the SLL, in the Transmitter, a $1 \times 192$ row of data is transformed into a $24 \times 8$ matrix. Upon reaching the Receiver, the data is then reshaped back into the original $1 \times 192$ row for further computation.

Data Encoding and Decoding: To mitigate transmission errors, we use Hamming code in the Transmitter to encode each row's 8 elements (64 bits). This encoding adds 8 bits of parity, resulting in a total of 72 bits, or 9 elements. The Receiver then performs decoding to retrieve the original 8 elements.

To further enhance reliability, pipeline registers are inserted along the cross-die paths, with synchronization achieved using clock pulses. Each clock cycle transmits one encoded row of data (72 bits), which effectively eliminates timing issues and data transmission errors, ensuring smooth data flow without sacrificing performance.

### 4.5. Ping-Pang matrix transposition module

In the self-attention mechanism, the $\boldsymbol{K}$ matrix needs to be transposed, and it is generated on-line without pretreatment. To reduce the latency cost of matrix transposition, we designed a Ping-Pang Transposition Module, as shown in Fig. 7. The overall operation consists of two modes: Mode 0 and Mode 1. In Mode 0, the Decision Unit labels the input matrix as 0, then the data of the matrix labeled as 0 is written row-by-row into URAM0. At the same time, the data in URAM1 is already full and is being output column-by-column as a transposed matrix. Once the matrix labeled 0 fills URAM0, the data in URAM1 has been completely read out. At this point, the Ping-Pang Matrix Transposition Module switches from Mode 0 to Mode 1. In Mode 1, the Decision Unit labels the input matrix as 1, and the data of the matrix labeled as 1 is written row-by-row into URAM1. Meanwhile, the data in URAM0 is already full and is being output column-by-column as a transposed matrix. Once the matrix labeled as 1 fills URAM1, the data in URAM0 has been completely read out. At this point, the Ping-Pang Matrix Transposition Module switches from Mode 1 back to Mode 0. This alternating cycle ensures seamless and non-blocking matrix transposition.

### 4.6. Matrix computation module

Matrix multiplication (MM) computation is the most computationally intensive operation in ViT, which is mainly in the self-attention mechanism and the Multi-Layer Perceptron (MLP) sub-layers, consuming most of the time and resources [40–42]. To accelerate the MM operation, Google's TPU proposes the most famous systolic array, executing the matrix multiplication with a fine-grained pipeline, thus it can reach great high frequency up to GHz. However, the proposed systolic array takes up too many computing resources and thus is not suitable for edge devices. To decrease the circuit area, we present the cyclical row systolic array (CRSA). Unlike the TPU or broadcast-style systolic arrays, the hardware computing array in CRSA consists of a single row of the matrix. In this pipeline, each row of the matrix is transmitted to all processing elements in the computing array at each clock cycle, while another matrix is transmitted synchronously to the computing array for multiplication, row by row. This process outputs a row of the resulting matrix after a few clock cycles. For ViT-tiny, consider the multiplication of the Key weights and the input as an example. The size of the Key weights is (192, 192), the size of the input is (192, 1), and the size of the processing element (PE) array is (192, 1). Each PE uses two DSP hardware units: one is used for multiplication between INT 8 and INT 8, and the other is used for result accumulation. In the first clock, the first element of the input is transmitted to all PE in the array, at the same time, the first row of weights of Key is transmitted to PE in the array respectively, and the PE performs multiplication and addition operations. In the second clock, the second element of the input is transmitted to all PE in the array, at the same time, the second row of weights of Key is transmitted to PE in the array respectively, and the PE performs multiplication and addition operations. After 192 clocks, the first row of the result matrix is calculated. After 24 redundant clock cycles, the vector (192,1) is transferred to the next module and receives new inputs (192,1).

To prevent overflow in our integer-only quantization scheme, the data bit-width in MM computation has been carefully designed. Matrix multiplication computation involves both the multiplication and accumulation parts of the matrix, meaning that the multiplication results need to be continuously accumulated. For example, in ViT-tiny, multiplying two 197x197 matrices requires accumulating 197 INT8 data items. Such accumulations can result in very large numbers, posing a risk of INT8 data overflow. To avoid this, we use INT16 for the accumulation. Here, our definition of INT16 is: the first bit is the sign bit, $2^{nd}$-$10^{th}$ bits are the integer part, and $11^{th}$-$16^{th}$ bits are the fractional part. Thus, multiplying two INT8 data items directly yields INT16 data, facilitates accumulation calculations. The largest number that can be represented by INT16 is binary number "0111111111111111", which is approximately 511.98 in decimal. This range supports the required multiplication calculations, effectively mitigating the risk of data overflow. Subsequently, the INT16 data may undergo further subtraction, division, and other operations, reducing its absolute value. When the absolute value of the INT16 data falls within the INT8 range ($\pm 15.875$), we truncate it to convert the data back to INT8. The specific operation involves selecting the first bit and $7^{th}$-$13^{th}$ bits of the INT16 data for bit concatenation. This operation discards redundant bits, resulting in INT8 data.

**Table 2**
Hardware utilization of DRViT on Alveo U250. LUTs, REGs, BRAMs, URAMs, and DSP slices are considered to characterize the hardware utilization of the FPGA Alveo U250. The hardware utilization for one layer can be divided into six parts: Linear, Attention, LayerNorm, GELU, Adder, and Global. The total hardware utilization includes four layers (one for each SLR), an SLR Management unit, an input unit, and an output unit. Taking into account the total hardware resources of the Alveo U250 (1,341K LUTs, 2,749K FFs (REGs), 2,000 BRAMs, 1,280 URAMs, and 11,508 DSP slices), the total utilization can be calculated.

| Component | LUT | REG | BRAM | URAM | DSP |
|---|---|---|---|---|---|
| Linear | 49.6k | 97.2k | 171.5 | 0 | 1728 |
| Attention | 75.4k | 70.5k | 0.5 | 45 | 965 |
| LayerNorm | 3.5k | 7k | 0 | 0 | 10 |
| GELU | 1.7k | 12.4k | 1 | 0 | 0 |
| Adder | 3.1k | 3.1k | 0 | 44 | 0 |
| Global | 1.6k | 76.8k | 0 | 0 | 0 |
| **Sum (one layer)** | **134.9k** | **267k** | **173** | **89** | **2,703** |
| Sum (all layers) | 539.6k | 1,068k | 1,903 | 356 | 10,812 |
| SLR Management | 4k | 12.8k | 0 | 0 | 0 |
| input | 1k | 3.1k | 0 | 0 | 0 |
| output | ≈0k | ≈0k | 0 | 0 | 0 |
| **Total** | **544.6k (40.61%)** | **1,083.9k (39.43%)** | **1,903 (95.15%)** | **356 (27.81%)** | **10,812 (93.95%)** |

### 4.7. Storage architecture

High energy efficiency is critical for edge device deployment and is primarily based on the following formula:

$$\text{Energy Efficiency} = \frac{\text{Operations}}{\text{Energy}}. \tag{6}$$

The energy consumption is given by:

$$\text{Energy} = M_{\text{DRAM}} E_{\text{DRAM}} + M_{\text{buffer}} E_{\text{buffer}} + \text{O} E_{\text{operation}}. \tag{7}$$

Energy Efficiency refers to the number of operations that can be performed per unit of energy. For a given algorithm, the number of operations is fixed. The energy consumption is primarily determined by computation and data movement, as shown in Eq. (7). Here, $E_{\text{DRAM}}$, $E_{\text{buffer}}$, and $E_{\text{op}}$ represent the energy consumed per access to DDR, on-chip buffer, and per operation, respectively. $M_{\text{DRAM}}$, $M_{\text{buffer}}$, and $O$ denote the number of accesses to DDR, on-chip buffer, and the number of operations in the algorithm, respectively. The energy required for accessing off-chip memory is greatly higher than that required for accessing the on-chip buffer or performing computations [14]. Thus, the primary factor influencing energy efficiency is the number of off-chip memory accesses. To minimize off-chip memory accesses, we propose a novel storage-optimized strategy to reduce resource occupancy. Storage data is divided into two categories: (a) $Weights$: have a larger storage size and include the weights necessary for calculating the Attention and Linear blocks. (b) $ActivationValues$: have a smaller storage size and include the activation values of the nonlinear parts of GELU and Softmax. This storage-optimized strategy fully utilizes and allocates three types of storage resources on the Alveo U250 FPGA for these data categories: Distributed RAM (DRAM), Block RAM (BRAM), and UltraRAM (URAM). Each type of storage resource has distinct advantages. BRAM, with its large storage array and wide transmission bit width, is primarily used for storing weights. In our design, it is configured as a ROM to store weights for Key ($\mathbf{W}_k$), Query ($\mathbf{W}_q$), Value ($\mathbf{W}_v$), and other weights, facilitating efficient output for matrix calculations. URAM, known for its high density, large capacity, and rapid read/write speeds, is flexibly configured to adapt to various application requirements. It serves as a high-speed data buffer for matrix flow and operations like matrix transposition. DRAM, the most flexible of the three storage resources with the smallest storage arrays, effectively reduces redundancy in data storage. In our design, DRAM stores Activation Values as a lookup table. We employ a non-uniform quantization method for the lookup table, using higher accuracy for smaller values and lower accuracy for larger values, ensuring consistent relative quantization error for different activation values. Based on this strategy, we designed the data flow for storage resources.

For example, in the attention module, as depicted in Fig. 2, the $\boldsymbol{K}$ matrix must be transposed after calculation and cannot be computed synchronously with the $\boldsymbol{Q}$ and $\boldsymbol{V}$ matrices. Thus, the input $x$ vector array is first buffered in URAM for the calculation of $\boldsymbol{Q}$ and $\boldsymbol{V}$. Due to the non-time order of matrix transposition in hardware, the entire matrix must be obtained before transposition can occur. Therefore, URAM buffers the $\boldsymbol{K}$ matrix for transposition. Because the output of $\boldsymbol{K}$ matrix transposition and Softmax occurs later, the $\boldsymbol{V}$ matrix is precomputed and cached in URAM, facilitating matrix multiplication with the Softmax output to produce the final attention output. Additionally, the weights $\boldsymbol{W}_k$, $\boldsymbol{W}_q$, and $\boldsymbol{W}_v$ in attention are pre-stored in BRAM, with $\boldsymbol{W}_q$ and $\boldsymbol{W}_v$ integrated to reduce BRAM resource redundancy. For the nonlinear exponential operations in Softmax and GELU, a lookup table method is employed to improve latency. Since only 256 INT8 quantized values are stored, DRAM, with its small storage array, is used. In the LayerNorm module, URAM buffers data to calculate the mean and variance. By effectively allocating and utilizing these three storage resources, a full-process on-chip data flow is achieved, considerably reducing communication latency and power consumption.

## 5. Evaluation

### 5.1. FPGA implementation

In this section, we present the implementation of DRViT on the Xilinx Alveo U250 FPGA. This FPGA features 1,341K LUTs, 2,749K FFs (REGs), 2,000 BRAMs, 1,280 URAMs, and 11,508 DSP slices. We measure the overall performance of DRViT by running the ViT-tiny model on ImageNet and conduct a comprehensive comparison with the performance of CPU, GPU, and other FPGA accelerators. The details are as follows.

We use Xilinx Vivado 2020.1 and 2022.1 to synthesize and deploy the complete project. The target platform is the Xilinx Alveo U250, used to implement our DRViT to run the ViT model. Fig. 8(c) shows the layout of our implementation on the Alveo U250. Due to the limitations and critical paths for timing closure associated with cross-die connections, we implement one layer on each SLR, and the SLR management, input, and output components are represented by the dark blue sections in the implementation. Fig. 8(a) illustrates the layout of our implementation in each layer. Note that Fig. 8(a) is not part of Fig. 8(c) but is included for demonstration purposes. Table 2 summarizes the resource utilization on the Alveo U250 FPGA board. We carefully designed the on-chip data flow from a global perspective and allocated resources efficiently, resulting in a BRAM utilization rate of 95.15%. Additionally, the heterogeneous Vector Systolic Array maximizes the use of DSP resources, achieving a DSP utilization rate of 93.95%.

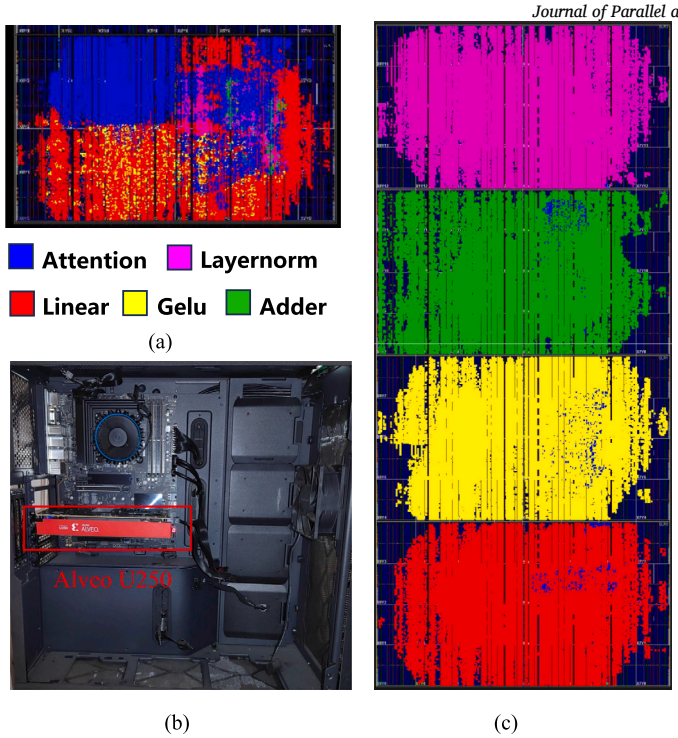


**Fig. 8.** Implementation on FPGA. (a). Illustrates the layout of our implementation for each layer. (b). The experimental operating environment with Alveo u250. (c). The layout of our implementation on the Alveo U250. Layout 4 layers total.

**Table 3**
Utilization of DSP and LUT compared with previous works. The design takes full advantage of the DSP (93.95%) to improve hardware utilization by stream pattern and on-chip flow.

| Related Work | DSP utilization | LUT utilization |
|---|---|---|
| Trans'23 [11] | 50.3% | 52.7% |
| HPCA'23 [43] | 78.1% | 50.4% |
| TCAD'22 [36] | 40.6% | 29.6% |
| **DRViT (ours)** | **93.95%** | **40.61%** |

### 5.2. Comparison with CPU and GPU

This section compares the performance and energy efficiency of our FPGA prototype with those of a CPU and GPU. The CPU baseline and GPU baseline are from ViA [36]. The CPU baseline is an Intel Core i7-5930K processor with a 15-MB cache, six physical cores, 12 threads operating at 3.7 GHz, and a thermal design power (TDP) of 140 W. The GPU baseline is an NVIDIA Tesla V100 with 32-GB GDDR5 and 5120 hardware threads operating at 1455 MHz.

Compared to the CPU baseline, our FPGA prototype achieves a 209.5× speedup in performance. Compared to the GPU, the performance improvement is 2.9×. In terms of energy efficiency, our FPGA prototype achieves a 312.3× speedup over the CPU and a 21.4× improvement over the GPU. Our design fully leverages the low power consumption of the FPGA.

### 5.3. Comparison with previous work

Table 4 presents a comparison with related works. Leveraging the lightweight hardware-friendly algorithm and novel specialized hardware architecture, our design achieves a high throughput of up to 1196 GOP/s and a low latency of 3.68 ms at a frequency of 140 MHz.

In the work of [11], a one-DSP-for-two INT8 multiplication scheme is employed to fully utilize the available DSP resources and achieve high performance. The well-defined stream dataflow in the hardware design allows for an operation frequency of up to 300 MHz. But this design exhibits excessively high latency, measured at 4.077 ms. ViA [36] proposes a novel ViT accelerator architecture based on FPGA and uses a proper partition strategy based on the data structure in the ViT, achieving 309.6 GOP/S computing performance, which is lower than ours. FTRANS [47] uses the FFT/IFF method to reduce the matrix calculation complexity. Some researchers have taken a different approach and optimized the nonlinear function in the calculation process of the Transformer to improve performance. Zhang [45] proposes a method that can compress the attention mechanism by 95%. Based on algorithm-hardware co-design, they obtain a run-time computing throughput of 1.87 Tops. However, the practical throughput performance is only 190.1 GOP/s. HeatViT [43] proposes a hardware-efficient image-adaptive token pruning framework. Although their work shows higher network accuracy, the hardware efficiency is lower than ours though pruning is considered. FT [48] uses a mixed block and vector-wise method for pruning, further reducing the computational complexity of the model. SWAT [46] proposes a novel tiling strategy to eliminate the redundant computations in shifted windows. It also deploys a dynamic pipeline interleaving dataflow that not only reduces processing latency but also maximizes data reuse, thereby decreasing memory access. While the practical end-to-end throughput performance is not that high, with only 301.9 GOP/s. The design of NPE [17] is the overlap. This pattern makes BRAM consumption less than the stream, but the cost is difficult to achieve high performance. The inference time in NPE is 232.37 ms when performing the BERT with sequence length 512. The throughput arrival is approximately the level of 100 GOPs. ELSA [49] redesigned a hardware-friendly similarity measurement method to replace the Softmax computing unit to improve overall performance. H. Peng [50] introduces an efficient Transformer acceleration framework for FPGA applications and proposes a column-balanced block-wise pruning method that maintains low accuracy decay under high pruning ratios. They design a specialized processing element for sparse matrix multiplication to enable inter-block hardware parallelism. However, the sparse or pruning scheme shows relatively weak generalization ability and requires a complex network training process. M. Sun [51] proposes an FPGA-aware automatic

**Table 4**
Performance comparison with previous works. We evaluated the performance of our work at 100 MHz and 140 MHz, respectively. Trans'23 [11], HPCA'23 [43], DAC'23 [44] employ similar models (ViT-tiny) and data types with our work. By way of dynamic redundancy-aware, the design achieves a 2.24× higher throughput and a 4.74× lower latency over previous state-of-the-art accelerators at 140 MHz.

| Related Work | Model | Data type | Freq. (MHz) | Throughout (GOP/s) | Energy efficiency (GOP/J) | latency (ms) |
|---|---|---|---|---|---|---|
| Trans'23 [11] | ViT-tiny | W8A8 | 300 | 616.1 | 20.8 | 4.077 |
| HPCA'23 [43] | ViT-tiny | W8 | - | 203 | 21.5 | 3.69 @1.74x Pruning |
| TCAD'22 [36] | Swin T | F16 | 300 | 309.6 | 7.9 | - |
| TECS'21 [45] | Multi30K | W8A8 | 125 | 190.1 | 8.4 | 6.8 |
| ASP-DAC'24 [46] | Swin T | W8 | 200 | 301.9 | 21.04 | - |
| FPGA'21 [17] | BERT-B | W16 | 200 | 207.94 | 10.40 | - |
| DAC'23 [44] | ViT-tiny | W8 | - | 271 | 28.7 | - |
| **DRViT (ours)** | **ViT-tiny** | **W8A8** | **140** | **1380.8** | **40.5** | **0.86** |
| | | | **100** | **986.3** | **38.9** | **1.17** |

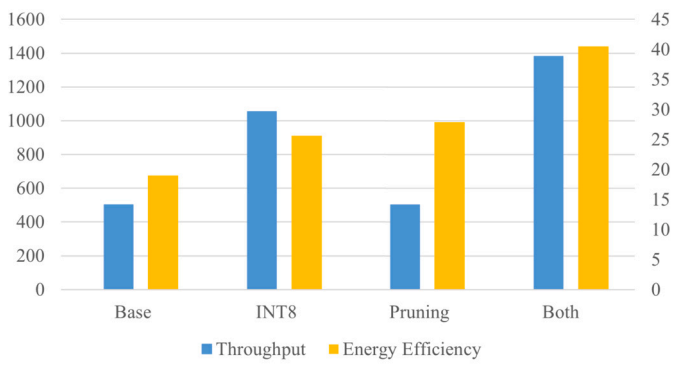


**Fig. 9.** The throughput and energy efficiency of base, INT8 quantization only, token merging and pruning only, and combined (INT8 quantization + token merging only and pruning). Combining two methods together significantly improves energy efficiency.

ViT acceleration (Auto-ViT-Acc) framework for a mixed-scheme ViT algorithm (low-bit with PoT quantization). Although it achieves higher throughput than ours, the mixed quantized network sacrifices accuracy and complicates the training process, particularly for large networks.

Table 3 shows the utilization of DSP and LUT compared with previous works. The hardware efficiency of DRViT is clearly higher than that of other designs. This efficiency stems from two main aspects. First, we employ the stream pattern instead of the control pattern, implementing different hardware units for each computing part of the target algorithm and optimizing them separately. This approach facilitates the full use of computing resources and allows for the optimization of parallel schemes in each hardware kernel. Second, our design minimizes off-chip memory accessing, which enables all computing units to run almost continuously without waiting for data from off-chip memory.

To explain more clearly how the proposed methods contribute to the observed performance gains, we break down the contribution of token pruning versus INT8 quantization, as shown in Fig. 9. It shows that the exclusive use of INT8 significantly reduces computational complexity, leading to faster processing speeds, lower power consumption, and improved energy efficiency. Additionally, INT8 contributes to data compression, which decreases the storage and communication resources required, thereby enhancing throughput. Although using TP alone does not directly impact throughput, it does reduce computational complexity after pruning, which results in some power consumption optimization. When combined, the hardware-friendly characteristics of INT8 further support the hardware circuits involved in TP, leading to additional reductions in power consumption. As a result of these power efficiency improvements, throughput is also enhanced. Overall, the synergy between INT8 and TP at the algorithmic level greatly increases both throughput and energy efficiency.

For energy efficiency, our work achieves a 2×-3× improvement over most existing approaches. The calculation of energy efficiency is based on Eq. (7), where the energy consumption of accessing off-chip memory significantly exceeds that of computation and accessing on-chip buffers. Therefore, in our design, we minimize the number of off-chip memory accesses by pruning redundant tokens and applying quantization techniques, as well as by designing a dedicated hardware architecture. These optimizations fully leverage the low-power characteristics of FPGAs, thereby enhancing energy efficiency.

## 6. Limitation and future work

It is essential to acknowledge the potential limitations and challenges in adapting these approaches to diverse hardware platforms or model types. Differences in architecture, memory hierarchy, and computational resources across platforms could impact the efficiency and effectiveness of these optimizations. Additionally, the specific characteristics of different models, such as attention mechanisms or layer configurations, may require tailored adjustments to fully leverage these techniques. This work, to some extent, sacrifices flexibility in exchange for achieving higher performance and energy efficiency. Developing specialized accelerators tailored to specific algorithms entails significant engineering effort. Although an initial plan was proposed to introduce an instruction set for broader applicability, it was ultimately abandoned due to latency and power consumption concerns. In the future, we aim to integrate this work with reconfigurable dataflow architectures to enhance its flexibility and scalability. Furthermore, the current design only supports on-chip inference. We plan to extend it to support on-chip training by reusing existing acceleration modules. Addressing these challenges will make this work suitable for deployment in robotics, leveraging its low power consumption and incremental learning capabilities to advance the development of embodied intelligence. Addressing these challenges is critical to ensuring the broad applicability and robustness of the proposed methodologies.

## 7. Conclusion

Given the limited computational capacity of edge devices and the substantial computational demands of Vision Transformers (ViT), we have proposed a novel dynamic redundancy-aware ViT accelerator, termed DRViT. DRViT is the result of a co-design approach that with both algorithm and architecture optimization. Firstly, we introduce a lightweight hardware-friendly algorithm that incorporates token merging, token pruning, and an INT8 quantization scheme, thereby reducing the need for floating-point operations. This algorithm is specifically tailored to enhance the performance and efficiency of ViT. Secondly, we design a specialized architecture that supports this algorithm, transforming it into significant improvements in latency and energy efficiency. This architecture has been implemented on the Xilinx Alveo U250 FPGA, demonstrating an overall inference latency of 0.86 ms and 1.17 ms per image for ViT-tiny at operating frequencies of 140 MHz and 100 MHz, respectively. The throughput peaks at 1,380 GOP/s. To the best of our

knowledge, DRViT outperforms other FPGA-based accelerators, achieving higher performance and lower latency at reduced frequencies. This work sets a new benchmark in the field, demonstrating the potential of co-design methodologies in developing efficient and high-performance ViT accelerators for edge devices.

## CRediT authorship contribution statement

**Xiangfeng Sun:** Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation. **Yuanting Zhang:** Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Qinyu Wang:** Writing – review & editing, Conceptualization. **Xiaofeng Zou:** Writing – review & editing. **Yujia Liu:** Writing – review & editing, Supervision. **Ziqian Zeng:** Writing – review & editing. **Huiping Zhuang:** Writing – review & editing, Visualization, Supervision, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This research was supported by National Natural Science Foundation of China (Grant No. 62406114, Grant No. 62306117, Grant No. 62472181), by the Guangzhou Basic and Applied Basic Research Foundation (Grant No. 2023A04J1687, Grant No. 2024A04J3681), by the Guangdong Basic and Applied Basic Research Foundation (Grant No. 2024A1515010220), by South China University of Technology-TCL Technology Innovation Fund, by the Fundamental Research Funds for the Central Universities (Grant No. 2024ZYGXZR074, Grant No. 2023ZYGXZR023), by the Postdoctoral Fellowship Program of CPSF (Grant No. GZC20230841), by the China Postdoctoral Science Foundation (Grant No. 2024M760955), and Sponsored by CAAI-MindSpore Open Fund, developed on OpenI Community.

## References

[1] W. Kim, B. Son, I. Kim, ViLT: vision-and-language transformer without convolution or region supervision, in: International Conference on Machine Learning, PMLR, 2021, pp. 5583–5594.

[2] A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning transferable visual models from natural language supervision, 18–24 Jul 2021.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: transformers for image recognition at scale, in: 9th International Conference on Learning Representations, 2020.

[4] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[5] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, H. Jégou, Training data-efficient image transformers & distillation through attention, in: International Conference on Machine Learning, PMLR, 2021, pp. 10347–10357.

[6] L. Zhao, Z. Zeng, Dap-SiMT: divergence-based adaptive policy for simultaneous machine translation, Int. J. Mach. Learn. Cybern. (2024) 1–20.

[7] X. Sun, Y. Zhang, Y. Jiang, Z. Li, B. Han, J. Mai, Z. Luo, E. Yao, HLC: a hardware-friendly quantization and cache-based accelerator for transformer, in: 2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS), IEEE, 2024, pp. 447–451.

[8] M.S. Akhtar, D.S. Chauhan, A. Ekbal, A deep multi-task contextual attention framework for multi-modal affect analysis, ACM Trans. Knowl. Discov. Data 14 (3) (2020) 1–27.

[9] S. Mahmud, X. Lin, J.-H. Kim, H. Iqbal, M. Rahat-Uz-Zaman, S. Reza, M.A. Rahman, A multi-modal human machine interface for controlling a smart wheelchair, in: 2019 IEEE 7th Conference on Systems, Process and Control (ICSPC), IEEE, 2019, pp. 10–13.

[10] I. Djitog, H.O. Aliyu, M.K. Traoré, A model-driven framework for multi-paradigm modeling and holistic simulation of healthcare systems, Simulation 94 (3) (2018) 235–257.

[11] M. Huang, J. Luo, C. Ding, Z. Wei, S. Huang, H. Yu, An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge, IEEE Trans. Circuits Syst. I, Regul. Pap. (2023).

[12] X. Sun, H. Tang, B. Han, Q. Chen, Z. Chang, L. Liu, K. Liu, Z. Hu, Y. Zhang, Y. Jiang, Optimized routing in multi-fpga systems: a congestion mitigation and tdm allocation approach, in: Ninth International Symposium on Advances in Electrical, Electronics, and Computer Engineering (ISAEECE 2024), vol. 13291, SPIE, 2024, pp. 1244–1249.

[13] J. Liu, B. Zhuang, Z. Zhuang, Y. Guo, J. Huang, J. Zhu, M. Tan, Discrimination-aware network pruning for deep model compression, IEEE Trans. Pattern Anal. Mach. Intell. 44 (8) (2022) 4035–4051.

[14] H. Jia, W. Deng, P. Guan, Z. Wang, B. Chi, 20.3 a 60ghz 186.5 dbc/hz fom quad-core fundamental vco using circular triple-coupled transformer with no mode ambiguity in 65nm cmos, in: 2021 IEEE International Solid-State Circuits Conference (ISSCC), vol. 64, IEEE, 2021, pp. 1–3.

[15] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo, Swin transformer: hierarchical vision transformer using shifted windows, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 10012–10022.

[16] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al., Language models are unsupervised multitask learners, OpenAI Blog 1 (8) (2019) 9.

[17] H. Khan, A. Khan, Z. Khan, L.B. Huang, K. Wang, L. He, NPE: an FPGA-based overlay processor for natural language processing, in: Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2021.

[18] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, L. Yuan, TinyViT: fast pretraining distillation for small vision transformers, in: European Conference on Computer Vision, Springer, 2022, pp. 68–85.

[19] W. Zeng, S. Jin, W. Liu, C. Qian, P. Luo, W. Ouyang, X. Wang, Not all tokens are equal: human-centric visual analysis via token clustering transformer, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 11101–11111.

[20] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, J. Hoffman, Token merging: your vit but faster, in: The Eleventh International Conference on Learning Representations, 2022.

[21] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, C.-J. Hsieh, DynamicViT: efficient vision transformers with dynamic token sparsification, Adv. Neural Inf. Process. Syst. 34 (2021) 13937–13949.

[22] H. Yin, A. Vahdat, J.M. Alvarez, A. Mallya, J. Kautz, P. Molchanov, A-ViT: adaptive tokens for efficient vision transformer, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 10809–10818.

[23] C. Li, Y. Peng, G. Liu, Y. Li, X. Yang, L. Zhang, C. Chen, Efficient vision transformer for human-centric aiot applications through token tracking assignment, IEEE Trans. Consum. Electron. (2023).

[24] H.-Y. Wang, T.-S. Chang, Row-wise accelerator for vision transformer, in: 2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS), IEEE, 2022, pp. 399–402.

[25] Z. Lit, M. Sun, A. Lu, H. Ma, G. Yuan, Y. Xie, H. Tang, Y. Li, M. Leeser, Z. Wang, et al., Auto-ViT-Acc: an FPGA-aware automatic acceleration framework for vision transformer with mixed-scheme quantization, in: 2022 32nd International Conference on Field-Programmable Logic and Applications (FPL), IEEE, 2022, pp. 109–116.

[26] M. Dhouibi, A.K. Ben Salem, A. Saidi, S. Ben Saoud, Accelerating deep neural networks implementation: a survey, IET Comput. Digit. Tech. 15 (2) (2021) 79–96.

[27] W. Lou, L. Gong, C. Wang, Z. Du, X. Zhou, OctCNN: a high throughput FPGA accelerator for CNNs using octave convolution algorithm, IEEE Trans. Comput. 71 (8) (2021) 1847–1859.

[28] S.I. Venieris, A. Kouris, C.-S. Bouganis, Toolflows for mapping convolutional neural networks on FPGAs: a survey and future directions, ACM Comput. Surv. 51 (3) (2018) 1–39.

[29] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, J. Cong, Optimizing FPGA-based accelerator design for deep convolutional neural networks, in: Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2015, pp. 161–170.

[30] Y. Sun, S. Wang, Y. Li, S. Feng, X. Chen, H. Zhang, X. Tian, D. Zhu, H. Tian, H. Wu, Ernie: Enhanced representation through knowledge integration, Computing Research Repository, 2019.

[31] Y. Liang, C. Ge, Z. Tong, Y. Song, J. Wang, P. Xie, Not all patches are what you need: Expediting vision transformers via token reorganizations, Computer Research Repository, 2022.

[32] X. Liu, W. Wei, C. Liu, Y. Peng, J. Huang, J. Li, Real-time monocular depth estimation merging vision transformers on edge devices for aiot, IEEE Trans. Instrum. Meas. 72 (2023) 1–9.

[33] M. Zhu, Y. Tang, K. Han, Vision transformer pruning, in: 2021 ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2021.

[34] J. Yu, K. Prabhu, Y. Urman, R.M. Radway, E. Han, P. Raina, 8-bit transformer inference and fine-tuning for edge accelerators, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 3, 2024, pp. 5–21.

[35] Z. Li, Q. Gu, I-ViT: integer-only quantization for efficient vision transformer inference, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 17065–17075.

[36] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, H. Chen, ViA: a novel vision-transformer accelerator based on FPGA, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 41 (11) (2022) 4088–4099.

[37] T. Geng, T. Wang, A. Sanaullah, C. Yang, R. Patel, M. Herbordt, A framework for acceleration of cnn training on deeply-pipelined FPGA clusters with work and weight load balancing, in: 2018 28th International Conference on Field Programmable Logic and Applications (FPL), IEEE, 2018, pp. 394–3944.

[38] L. Gong, C. Wang, X. Li, H. Chen, X. Zhou, MALOC: a fully pipelined FPGA accelerator for convolutional neural networks with all layers mapped on chip, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 37 (11) (2018) 2601–2612.

[39] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 10684–10695.

[40] K. Li, J. Zhou, Y. Wang, J. Luo, Z. Yang, S. Yang, W. Mao, M. Huang, H. Yu, A precision-scalable energy-efficient bit-split-and-combination vector systolic accelerator for nas-optimized dnns on edge, in: 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE), IEEE, 2022, pp. 730–735.

[41] N.P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, et al., In-datacenter performance analysis of a tensor processing unit, in: Proceedings of the 44th Annual International Symposium on Computer Architecture, 2017, pp. 1–12.

[42] M. Huang, Y. Liu, C. Man, K. Li, Q. Cheng, W. Mao, H. Yu, A high performance multi-bit-width booth vector systolic accelerator for nas optimized deep learning neural networks, IEEE Trans. Circuits Syst. I, Regul. Pap. 69 (9) (2022) 3619–3631.

[43] P. Dong, M. Sun, A. Lu, Y. Xie, K. Liu, Z. Kong, X. Meng, Z. Li, X. Lin, Z. Fang, et al., HeatVit: hardware-efficient adaptive token pruning for vision transformers, in: 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), IEEE, 2023, pp. 442–455.

[44] Z. Li, Y. Xie, P. Dong, O. Chen, Y. Wang, Algorithm-software-hardware co-design for deep learning acceleration, in: 2023 60th ACM/IEEE Design Automation Conference (DAC), IEEE, 2023, pp. 1–4.

[45] X. Zhang, Y. Wu, P. Zhou, X. Tang, J. Hu, Algorithm-hardware co-design of attention mechanism on FPGA devices, ACM Trans. Embed. Comput. Syst. 20 (5s) (2021) 1–24.

[46] Q. Dong, X. Xie, Z. Wang, SWAT: an efficient swin transformer accelerator based on FPGA, in: 2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC), IEEE, 2024, pp. 515–520.

[47] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, C. Ding, FTRANS: energy-efficient acceleration of transformers using FPGA, in: Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design, 2020, pp. 175–180.

[48] P. Qi, E.H.-M. Sha, Q. Zhuge, H. Peng, S. Huang, Z. Kong, Y. Song, B. Li, Accelerating framework of transformer by hardware design and model compression co-optimization, in: 2021 IEEE/ACM International Conference on Computer Aided Design (ICCAD), IEEE, 2021, pp. 1–9.

[49] T.J. Ham, Y. Lee, S.H. Seo, S. Kim, H. Choi, S.J. Jung, J.W. Lee, ELSA: hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks, in: 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), IEEE, 2021, pp. 692–705.

[50] H. Peng, S. Huang, T. Geng, A. Li, W. Jiang, H. Liu, S. Wang, C. Ding, Accelerating transformer-based deep learning models on FPGAs using column balanced block pruning, in: 2021 22nd International Symposium on Quality Electronic Design (ISQED), IEEE, 2021, pp. 142–148.

[51] M. Sun, Z. Li, A. Lu, H. Ma, G. Yuan, Y. Xie, H. Tang, Y. Li, M. Leeser, Z. Wang, et al., FPGA-aware automatic acceleration framework for vision transformer with mixed-scheme quantization: late breaking results, in: Proceedings of the 59th ACM/IEEE Design Automation Conference, 2022, pp. 1394–1395.